

MLOps Pipeline for Tourism Package Prediction

Done by Mazin Alahmadi

August 29, 2026

Contents / Agenda

- Executive summary
- Business overview and solution approach
- Data registration and preparation
- Customer insights from tourism data
- Model building, tracking, and performance
- Deployment and automated GitHub workflow
- Output evidence, recommendations, and appendix

Executive Summary

- The tourism campaign has a 19.3% package-purchase rate, so better targeting can materially improve sales productivity.
- A completed MLOps pipeline now registers data, cleans and splits it, trains a tuned XGBoost model, and packages a deployable Streamlit app.
- The best local model achieved 91.3% test accuracy, 76.4% precision, 79.4% recall, and 77.8% F1 at the selected threshold.
- Business recommendation: prioritize high-probability customers first, then monitor conversion lift through the automated workflow.

Business Overview and Solution Approach

- Business problem: sales teams need to focus outreach on customers most likely to purchase a tourism package.
- Decision lens: rank prospects before calling so high-fit customers receive faster and better-timed follow-up.
- Solution: convert customer attributes into a purchase-probability score using a reproducible, deployment-ready ML workflow.
- Operating model: GitHub controls source code, Hugging Face stores data/model/app assets, and GitHub Actions runs the pipeline end to end.

Data Registration

- Master project folder: `tourism_project` with `data`, `model_building`, `deployment`, `hosting`, `reports`, and `models` subfolders.
- Raw data asset: `tourism_project/data/tourism.csv`, documented with a dataset README for governance.
- Hugging Face dataset target: `mazin903/tourism`
- Registration script creates or reuses the dataset repo and uploads the raw dataset files.
- Benefit: the workflow has a single governed data source instead of manual file movement.

Data Preparation

- Loaded the tourism dataset from Hugging Face when credentials are available, with local fallback for development.
- Removed generated index/customer ID fields, removed duplicates, and standardized inconsistent categorical values.
- Imputed missing numeric values with medians and categorical values with modes.
- Created a stratified 80/20 split: 3,208 training rows and 803 testing rows.
- Saved cleaned, train, and test files locally and prepared automatic upload back to the dataset repo.

Customer Insights That Drive Targeting

- Base purchase rate: 19.3%, confirming the need for selective campaign prioritization.
- Passport holders convert at 35.9% versus 12.4% for non-passport customers.
- Basic package prospects convert at 30.1%, the strongest product segment in the dataset.
- Single customers convert at 30.9%, more than double married and divorced segments.
- Recommendation: combine model scores with these high-conversion segments for call-list prioritization.

Model Building with Experimentation Tracking

- Model choice: XGBoost classifier with preprocessing for numeric and categorical fields.
- Class imbalance handled with `scale_pos_weight` so minority buyers are not ignored.
- Grid search evaluated 48 parameter/run combinations using F1 as the selection metric.
- MLflow logs parameters, metrics, artifacts, model path, and threshold analysis for comparison.
- Best model is packaged for Hugging Face Model Hub target: `mazin903/tourism-package-model`

Model Performance and Decision Threshold

- Selected threshold: 0.50, chosen for the best local F1 with a balanced precision/recall tradeoff.
- Test performance: 91.3% accuracy, 76.4% precision, 79.4% recall, 77.8% F1.
- Business meaning: most predicted buyers are actionable, while recall still captures nearly four out of five actual buyers.
- Top signals: Passport, Basic package, Single marital status, Executive designation, and CityTier.
- Operational use: score prospects, prioritize top-probability leads, and monitor uplift after deployment.

Model Deployment

- Deployment surface: Streamlit app on Hugging Face Spaces, with Dockerfile configuration retained for portability.
- The app downloads the registered model bundle from Hugging Face Model Hub at startup.
- User inputs are converted into the same feature schema used during model training.
- Output: purchase probability, decision threshold, and a business-friendly follow-up recommendation.
- Hugging Face Space target: mazin903/tourism-package-prediction

Automated GitHub Actions Workflow

- Trigger: push to main or manual workflow dispatch.
- Pipeline steps: install dependencies, register dataset, clean/split data, generate analysis, train/register model, and deploy app.
- Secrets and variables separate credentials from source code: HF_TOKEN, HF_USERNAME, dataset repo, model repo, and Space repo.
- Generated processed data and reports are committed back to main with skip-ci protection.
- Benefit: one repeatable workflow replaces manual notebook-only execution.

Output Evaluation Evidence

- GitHub repository: <https://github.com/mazin903/MLOps-pipeline-on-GitHub>
- Hugging Face dataset: <https://huggingface.co/datasets/mazin903/tourism>
- Hugging Face model: <https://huggingface.co/mazin903/tourism-package-model>
- Streamlit app on Hugging Face Space: <https://huggingface.co/spaces/mazin903/tourism-package-prediction>
- Screenshots to capture after first account run: repo folder structure, green GitHub Actions workflow, Hugging Face dataset/model pages, and Streamlit app result.

APPENDIX

Dataset Overview and Registration Details

- Source file: tourism.csv with customer demographics, interaction history, product pitch attributes, and purchase outcome.
- Target variable: ProdTaken, where 1 means the customer purchased the package and 0 means they did not.
- Model features include age, income, city tier, follow-ups, satisfaction, passport, car ownership, occupation, package pitched, and designation.
- Processed outputs: cleaned_tourism.csv, Xtrain.csv, Xtest.csv, ytrain.csv, and ytest.csv.
- Registration plan: raw and processed files are uploaded to the Hugging Face dataset repo for reproducible downstream steps.

Experimentation Tracking Details

- Algorithm: XGBoost classifier selected from the assignment-approved model family.
- Tuning grid: n_estimators 100/200, max_depth 3/5, learning_rate 0.05/0.10, reg_lambda 1.0/5.0, colsample values 0.8.
- Best parameters: n_estimators 200, max_depth 5, learning_rate 0.10, reg_lambda 1.0.
- Tracked artifacts: metrics, classification report, threshold analysis, feature importance, and joblib model bundle.
- Model Hub registration stores the trained model and model card for deployment reuse.

GitHub Actions Workflow Details

- Repository-ready workflow: `.github/workflows/pipeline.yml`.
- Step 1: install Python dependencies and validate the project environment.
- Step 2: register raw data, prepare train/test assets, and generate business summary tables.
- Step 3: train the model, log results, and upload the best bundle to Hugging Face Model Hub.
- Step 4: push Streamlit deployment files to Hugging Face Spaces and commit generated reports back to main.